public final boolean isAlive() :

As we know when we call start() method then a new thread will be started in a separate Stack Memory.

Thread class has provided a predefined final non static method called isAlive(), return type is boolean.

This method is used to verify whether a thread has started or not OR the thread is alive or not.
[If a thread started and completed its asigned task then It is NOT alive]

If we use isAlive() method before the start method then it will return false because Thread has not started yet, on the other hand if we use isAlive() method, after start() method then it will return true but thread should be in running mode.

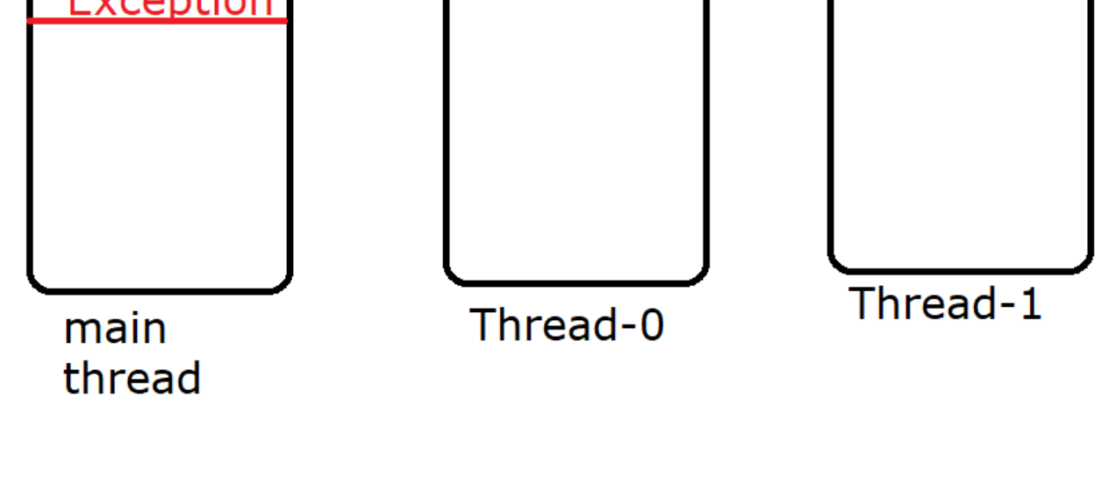
In java, We **cannot re-start a thread**, If we try to re-start then it will generate a runtime exception i.e java.lang.IllegalThreadStateException.

//Program

```java
package com.ravi.basic;

class Test extends Thread
{
    @Override
    public void run()
    {
        IO.println("Child thread is running");
        IO.println("It is  running in a separate Stack memory");
    }
}

public class IsAliveDemo
{
    public static void main(String[] args)
    {
       IO.println("Main Thread started!!!");

       Test t1 = new Test();
       IO.println("Is thread started before start :"+t1.isAlive());

       t1.start();

       IO.println("Is thread started after start :"+t1.isAlive());

       IO.println("Main Thread ended!!!");

       t1.start();  //java.lang.IllegalThreadStateException

     }
}
```

Exception in main thread :

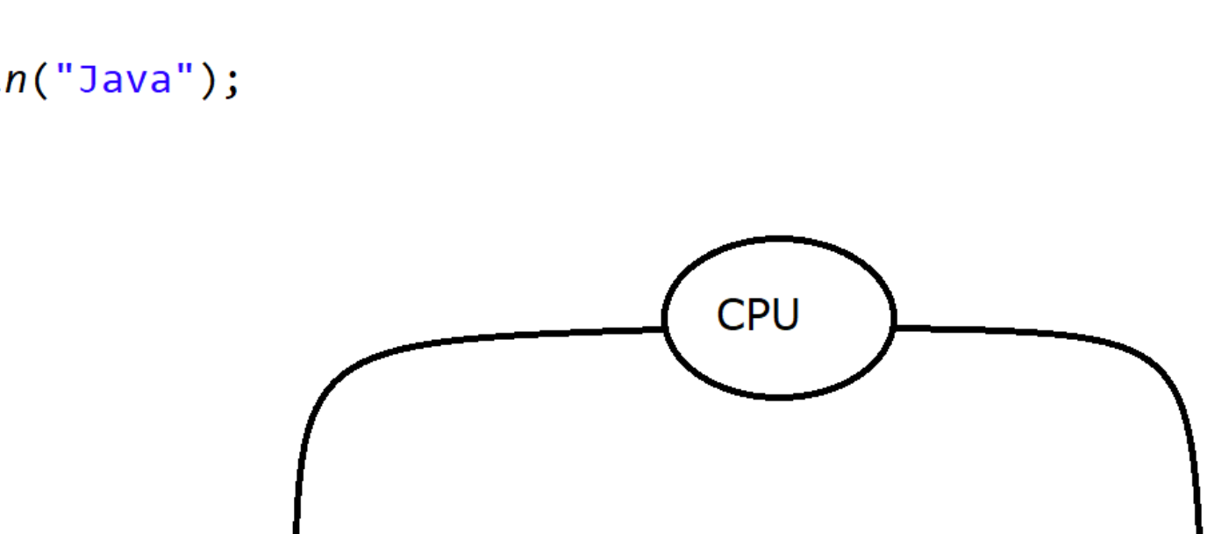
```java
 package com.ravi.basic;

 class Foo extends Thread
 {
     @Override
     public void run()
     {
         String name = Thread.currentThread().getName();
         IO.println("Running thread name is :"+name);
     }

 }
 public class ExceptionDemo
 {
     public static void main(String[] args)
     {
         IO.println("Main thread started");

         String name = Thread.currentThread().getName();
         IO.println("Current thread name is :"+name);

         Foo f1 = new Foo();
         f1.start();

         Foo f2 = new Foo();
         f2.start();

         IO.println(10/0);

         IO.println("Main thread ended");

     }

 }
```



Note :- Here main thread is interrupted due to AE but still child threads will be executed because child threads are executing with separate Stack.

ThreadLoop.java

```java
 package com.ravi.basic;

 class Sample extends Thread
 {
     @Override
     public void run()
     {
         String name = Thread.currentThread().getName();

         for(int i=1; i<=10; i++)
         {
             IO.println(i+" by "+name+" thread!!!");
         }
     }
 }
 public class ThreadLoop
 {
     public static void main(String[] args)
     {
         new Sample().start();

         String name = Thread.currentThread().getName();

         for(int i=1; i<=10; i++)
         {
             IO.println(i+" by "+name+" thread @");
         }

         int x = 1;

         do
         {
             IO.println("Java");
             x++;
         }
         while(x<=10);

     }

 }
```



Note : Here processor is frequently switching from main thread to Thread-0 thread so output is **un-predicatable.**

We have something called **Thread Schedular** which is responsible to schedule the thread that means it is schedular who will decide which thread will get the **processor time.**

Can a running thread will be suspended by Thread Schedular ?

Yes, because thread schedular is responsible to schedule the all the threads waiting for processor time so, all the threads will get the processor time.

Note : The Currently executing instruction of a running thread is stored in **PC register**.

How to set and get the name of the Thread :

Whenever we create a userdefined Thread in java then by default JVM assigns the name of thread is Thread-0, Thread-1, Thread-2 and so on.

If a user wants to assign some user defined name of the Thread, then Thread class has provided a predefined method called setName(String name) to set the name of the Thread.

On the other hand we want to get the name of the Thread then Thread class has provided a predefined method called getName().

```java
class Thread implements Runnable
{
  private volatile String name;

  public final synchronized void setName(String name) //setter
  {
    this.name = name;
  }

  public final String getName() //getter
  {
     return this.name;
  }
}
```

```java
 package com.ravi.thread_name;

 class Test extends Thread
 {
     @Override
     public void run()
     {
         String name = Thread.currentThread().getName();
         IO.println("Current thread name is :"+name);
     }
 }

 public class SettingAndGettingTheThreadName
 {
     public static void main(String[] args)
     {
        Test t1 = new Test();
        t1.setName("Child 1");


        Test t2 = new Test();
        t2.setName("Child 2");


        t1.start(); t2.start();
     }

 }
```

---

public final long threadId() :

* It is a new method which introduced from JDK 19V, Actually before threadId() method we had getId() method to get the id of the thread but this method is not final so this method is deprecated from JDK 19V and instead of getId() now we have threadId() method which is a final method.

* Both the methods are used to provide the id of the current thread. the threadId() method internally calling getId() method only.

```java
package com.ravi.thread_name;

class A extends Thread
{

}

public class ThreadID {

    public static void main(String[] args)
    {
        Thread thread = Thread.currentThread();
        IO.println("Main thread id is :"+thread.getId()); //getId() is deprecated JDK 19
        IO.println("Main thread id is :"+thread.threadId());

        A a1 = new A();
        IO.println("Id of child thread "+a1.threadId());

    }

}
```